Communication Design (servidor)

API REST del servidor: traduce peticiones HTTP a llamadas de la lógica de negocio. Notación: Logical Design & Reverse Engineering Specification v3.

Component Design

La capa de comunicación es la **frontera** con el exterior. Recibe HTTP, extrae y valida los datos, llama a la capa de negocio (`business_logic_design.md`) y construye la respuesta (código de estado + JSON). No contiene reglas de negocio.

```
   (cliente: app Android o web)
             |
             v   HTTP
   ------------ Communication (endpoints) ------------
   |                                                  |
   |  guardarMedicion.php       (POST)                |
   |  recuperarMedicion.php     (GET)                 |
   |                                                  |
   ----------------------------------+----------------
             |                       |
             | HTTP<->dominio        | HTTP<->dominio
             v                       v
        LogicaDeNegocio.guardarMedicion() / recuperarMedicion()
             |                       |
             v                       v
        ------------------ mediciones ------------------
```

Endpoints

Método	Ruta	Entrada	Salida
`POST`	`/src/communication/guardarMedicion.php`	cuerpo JSON `{ "id_plaquita", "tipo_medicion", "valor_medicion" }`	`200` `{ "ok": true }`
`GET`	`/src/communication/recuperarMedicion.php`	query `?desde&hasta&tipo_medicion&valor_medicion`	`200` `{ "mediciones": [ { "id_plaquita","tipo_medicion","valor_medic } ] }`

Códigos de estado

Situación	Código	Cuerpo
Operación correcta	`200 OK`	`{ "ok": true }` / `{ "mediciones": [ ... ] }`
Datos inválidos (JSON mal formado, tipo no permitido, valor ausente...)	`400 Bad Request`	`{ "error": "..." }`
Fallo de la base de datos	`500 Internal Server Error`	`{ "error": "..." }`

Traducción HTTP ↔ dominio

```
guardarMedicion.php (POST)
   |
   | 1. leer cuerpo crudo y parsearJSON( cuerpo ) --> objeto     (auxiliares.php)
   |        ? JSON mal formado -> ResponderHTTP( 400, error )
   |
   | 2. extraer los campos del objeto -> m: Medicion
   |
   | 3. LogicaDeNegocio.guardarMedicion( m ) --> ResultadoGuardado
   |        ? ok=false, clase='validacion'   -> ResponderHTTP( 400, error )
   |        ? ok=false, clase='persistencia' -> ResponderHTTP( 500, error )
   |        ? ok=true                        -> ResponderHTTP( 200, {ok:true} )

recuperarMedicion.php (GET)
   |
   | 1. extraerFiltros( $_GET ) --> f: FiltroMediciones          (auxiliares.php)
   |
   | 2. LogicaDeNegocio.recuperarMedicion( f ) --> ResultadoLista
   |        ? ok=false, clase='persistencia' -> ResponderHTTP( 500, error )
   |        ? ok=true                        -> ResponderHTTP( 200, {mediciones:[...]} )
```

El campo de dominio `clase` (`'validacion'` o `'persistencia'`, ver `business_logic_design.md`) es lo único que decide el código: el negocio nunca fija un `200`/`400`/`500`.

Funciones de la capa

Función	Entrada	Salida
`guardarMedicion()`	`POST` (cuerpo JSON)	respuesta HTTP
`recuperarMedicion()`	`GET` (query)	respuesta HTTP
`parsearJSON( cuerpo: Text )`	texto del cuerpo	`objeto` (o `null` si está mal formado)
`extraerFiltros( query )`	`$_GET`	`FiltroMediciones`

`parsearJSON()` y `extraerFiltros()` viven en `communication/auxiliares.php`, un fichero que **solo define funciones** (no atiende peticiones), para poder probarlas sin ejecutar un endpoint. El volcado de la respuesta (código + JSON) se hace inline en cada endpoint. Las funciones de la capa de negocio no aparecen aquí.

Diseño de funciones (notación v3)

La capa de comunicación no define clases: son **endpoints** (ficheros `.php` que atienden una petición) y **funciones auxiliares**. A la izquierda, `-->` apunta a la función cuando recibe datos y `<--` sale de ella cuando devuelve.

```
                 -------------- guardarMedicion.php --------------    // endpoint POST
                 |
                 |
cuerpo JSON --> guardarMedicion() --> respuesta HTTP
                 |
                 ---------------------------------------------------


                 -------------- recuperarMedicion.php -------------    // endpoint GET
                 |
                 |
query GET --> recuperarMedicion() --> respuesta HTTP
                 |
                 ---------------------------------------------------
```

```
                 ------------------ auxiliares.php ------------------
                 |
                 |
cuerpo: Text --> parsearJSON() --> Medicion?
                 |
                 |
query: [ Key: Text ] --> extraerFiltros() --> FiltroMediciones
                 |
                 ---------------------------------------------------
```

Función	Tipo	Entrada	Salida
`guardarMedicion()` (endpoint `POST`)	endpoint	cuerpo JSON	respuesta HTTP (`200`/`400`/`500`/`405`)
`recuperarMedicion()` (endpoint `GET`)	endpoint	query `GET`	respuesta HTTP (`200`/`500`/`405`)
`parsearJSON()`	pura (`--x`)	`cuerpo: Text`	`Medicion` o `null`
`extraerFiltros()`	pura (`--x`)	`query: [ Key: Text ]`	`FiltroMediciones`

Design Clarifications

- Cada fichero `.php` de `communication/` es un **endpoint autónomo**: arranca, hace su trabajo y responde. Ningún endpoint implementa reglas de negocio; toda validación de dominio vive en `business_logic_design.md`.
- El **canal de comunicación web es HTTP con JSON**; la capa de negocio no lo conoce. La conversión de `(codigo, cuerpo)` a dominio y viceversa se hace aquí.
- Un fallo de dominio por datos inválidos (`clase='validacion'`) se traduce a `400`; un fallo de la base de datos (`clase='persistencia'`) se traduce a `500`. El código de estado se decide **solo** aquí, nunca en la lógica de negocio.
- Las rutas son relativas a la raíz del servidor (`Sprint_0_Marc_Villalba`), por eso empiezan por `src/communication/`.